VeriReview

An Evidence-Based System for Verifying the Resolution of Source Code Review Comments

Project Report

Nandhagopal

[Department / Institution]

[Course / Supervisor]

14 October 2026

Abstract

A pull-request review comment is often marked resolved without any check that the underlying code change actually satisfies what the reviewer asked for. This report presents VeriReview, a system that verifies, from structural evidence rather than textual similarity, whether a code change made after a review comment satisfies the requirement expressed in that comment. The system reconstructs the window of commits following a comment, extracts one or more categorised requirements from the comment text, parses the resulting code into structural facts using an incremental parser, and applies a deterministic rule per requirement category to decide whether the requirement is met. A machine-learned code-similarity model is retained only as non-determinative supporting evidence, since evaluation showed that similarity scoring alone accepts a large fraction of incorrect fixes. Results are aggregated into one of four verdicts with a confidence value and a cited explanation, and a policy layer maps this to a recommended action, permitting an automated blocking action only once several independent, statistically grounded conditions are met. The system was evaluated on three successive benchmarks, each frozen before the rules it measured were changed and each run exactly once under a pre-registered protocol. A first round of rule refinement raised accuracy from 42% to 65% and halved the false-acceptance rate, to 15%, on 147 held-out cases. On the most recent 148-case test, whose real-world cases carry human labels, the system reached 57% accuracy with a false-acceptance rate of 9%. A second refinement round lowered the false-acceptance rate on development data from 6.1% to 0.7%, yet changed only 2 of the 148 held-out verdicts. This measured result shows that fixes written for individual error patterns transfer poorly to new cases. The system has also been run end to end as a GitHub App on a live test repository. The report sets out, separately and explicitly labelled as targets rather than measured results, the thresholds the complete system must reach before it may ever block a merge.

Contents

1. Introduction

2. Related Work

3. System Architecture

4. Verification Methodology

5. Integration with GitHub (Advisory Mode)

6. Dashboard and Operational Tooling

7. Evaluation and Results

8. Toward the Complete System: Targets and Projected Outcomes

9. Threats to Validity and Replication

10. Conclusion

References

1. Introduction

1.1 Problem Statement

In collaborative software development, a proposed change is examined through a code-review workflow before it is merged into a shared codebase. A reviewer leaves a comment identifying a defect or requesting a change at a specific location in a file. The author then pushes one or more further commits and marks the discussion thread as resolved. On most hosting platforms, marking a thread resolved is a purely administrative action taken by the author and is not checked against the actual content of the subsequent commits. A thread can therefore be resolved even where the requested change is missing, incomplete, made in the wrong place, or represented only by a comment or log message that mentions the fix without implementing it.

This report addresses the following question: given a review comment and the commits made after it, did the resulting code actually satisfy what the reviewer asked for?

1.2 Motivation

Review capacity is limited, and a resolved thread rarely receives further attention from the reviewer. An automated, evidence-backed second opinion on whether a resolution was genuine can surface the cases most worth a reviewer's remaining attention, without pretending to replace human judgement on matters that are inherently ambiguous.

1.3 Contributions

- A deterministic, request-level way of verifying review resolution: each request in a comment is checked against parsed program structure and every verdict cites file, lines and commit. The closest existing work asks a language model to read a textual diff (Section 2).
- A pipeline that reconstructs the exact window of code that changed after a review comment, including cases complicated by a rebase or a force-push.
- A rule-based verification approach, keyed to a requirement's category and grounded in parsed program structure, together with an empirical comparison against text-similarity and machine-learned baselines.
- A policy mechanism that gates any automated blocking action behind several independently configurable, statistically grounded conditions, so that blocking is only ever enabled once its safety has actually been measured.
- A full, working integration with a real hosting platform (signed webhooks, a durable job queue, least-privilege access tokens), exercised live on a test repository, and a read-only inspection dashboard.
- Three frozen, pre-registered blind evaluations, each run once, with human labels for the real-world cases, and an honest report of the resulting accuracy and error rates. These include a negative result: a second round of rule refinement did not transfer to new cases. Explicit, numeric targets for the complete system are stated separately (Section 8).

2. Related Work

2.1 Checking Whether Review Comments Were Addressed

The work closest to this project is Sun et al. (2025). That study asked whether comments from AI review tools on GitHub lead to code changes. It built a two-stage judge based on a large language model: the first stage decides whether a comment is valid, and the second labels a valid comment fully addressed, partially addressed, unaddressed or uncertain. The judge reads the reviewed code and a textual diff up to the merge commit. On 150 hand-annotated comments it reached 86.1% accuracy, and the human annotators agreed with a Cohen's kappa of 0.67 to 0.76. Its verdicts are close to the four used in this report. The task itself is therefore not new. That study uses no structural program analysis, does not discuss prompt injection, and chose its models on the same annotated sample it reports on.

Commercial review assistants offer a related feature. GitHub Copilot code review (2026) resolves one of its own comments when a later commit addresses it. CodeRabbit resolves its own open threads on its next review. Neither documents how the decision is made or reports error rates, and both apply only to comments the tool itself wrote. Earlier empirical work studied the outcome rather than verifying it. Bosu et al. (2015) and Rahman et al. (2017) predicted whether a review comment is useful, partly labelling usefulness by whether a change followed near the commented lines. Beller et al. (2014) classified review-induced changes and found that 7 to 35% of review comments are discarded.

2.2 Generating the Fix Instead of Checking It

A second line of work learns to produce the revised code from a review comment. Examples are Tufano et al. (2021), up to 31% of comments implemented correctly, CodeReviewer (Li et al., 2022), and Google's ML-based comment resolution (2023), about 52% of comments at 50% precision offline. These systems generate a candidate fix; none of them checks whether the fix a human actually made satisfies the comment, which is the question addressed here.

2.3 Similarity Scoring and Code Language Models

Another approach scores the similarity between a review comment and the subsequent change, using lexical overlap, TF–IDF weighting, sentence embeddings (Reimers and Gurevych, 2019), or a code model such as UniXcoder (Guo et al., 2022). Such scoring rewards a change that repeats the reviewer's request, in a comment, a log message or unreachable code, without implementing it. In this project's latest blind test, the code-model and structural baselines accepted 70 to 98% of incorrect fixes (Section 7). A code model is therefore used here only as non-determinative supporting evidence.

2.4 Structural Change Analysis and Comment Consistency

Tree-differencing tools such as ChangeDistiller (Fluri et al., 2007) and GumTree (Falleri et al., 2014; Falleri and Martinez, 2024) compute fine-grained edit scripts over syntax trees. This project also works on syntax trees, using Tree-sitter. It does not produce an edit script: it asks targeted questions of the code, such as whether a guard rejects the requested values before use, or whether a handler catches the named exception. Panthaplackel et al. (2021) detect, at commit time, when a code comment becomes inconsistent with changed code. That task concerns documentation inside the code, not a reviewer's request.

2.5 Security of Automated Review Agents

Reports from 2026 show that LLM-based review agents integrated with GitHub can be steered by attacker-controlled text in pull-request titles, comments and code comments, in some cases leaking credentials. This motivates the treat-content-as-data design of Section 4 and its measured robustness.

2.6 Positioning

Table 1 summarises how this work differs from the closest existing approaches. The contribution is not the task itself. It is a deterministic, structure-based and evidence-citing way of performing the task, built for safety, applied to any reviewer's comments, and evaluated more strictly.

Table 1: Comparison with the closest existing approaches.

	Sun et al. (2025)	**Copilot / CodeRabbit**	**This work**
Comments covered	AI and human comments	the tool's own comments	any reviewer's comments
How it decides	LLM reads a text diff	not disclosed	rules over parsed program structure
Evidence given	list of issues	none documented	file, lines and commit per request
When unsure	“uncertain” label	thread left open	“uncertain”; a human decides
Prompt injection	not discussed	not documented	0 of 2,576 planted variants changed an outcome
Evaluation	150 comments; models chosen on them	none published	three frozen, pre-registered blind tests
Can it block merges?	no (study)	approval gating (CodeRabbit)	only past a statistical gate; none passes today

3. System Architecture

Figure 1 shows the overall architecture. A source code hosting service supplies review threads and commit history, either on request or through a webhook-driven advisory interface. The remaining modules operate on a single, self-contained record of the review — the thread, the file before and after the relevant commits, and the diff between them — so that each stage can be reasoned about, and tested, independently of the others.

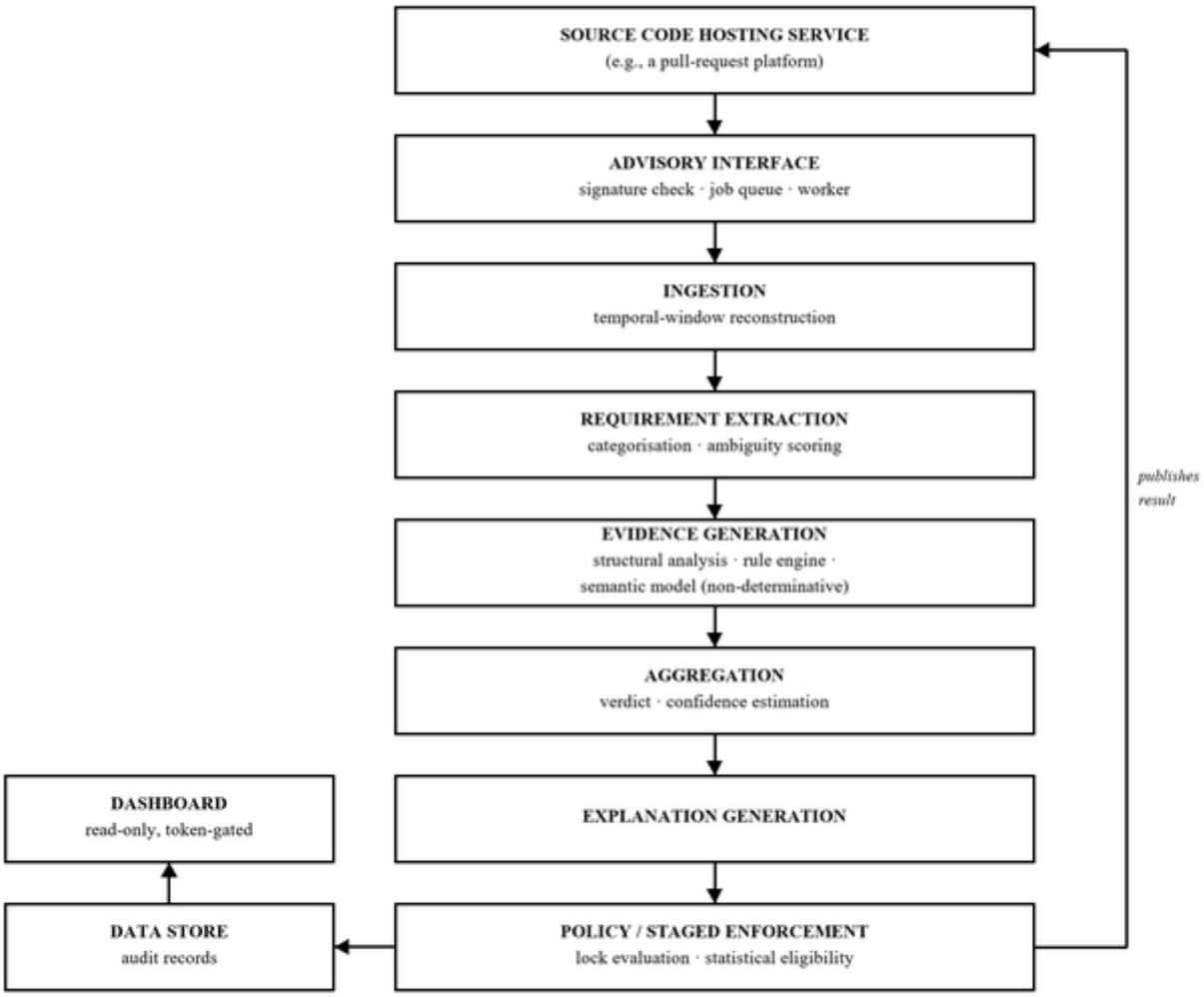


Figure 1: System architecture of VeriReview.

3.1 Ingestion and Temporal-Window Reconstruction

The ingestion stage identifies, for a given review comment, the sequence of commits made to the affected file after the comment and up to the point the pull request was resolved. This window must remain correct across a rebase or a force-push, both of which can rewrite the commit history of a pull request without changing what the reviewer actually asked for. Where the exact resolution time cannot be determined from the hosting service, the window is taken up to the head of the pull request, and this and other reliability concerns are recorded as flags that later reduce the confidence attached to a verdict.

3.2 Requirement Extraction

A review comment is split into one or more atomic requirements — a single action on a single target — since a comment such as “validate the input and return HTTP 400” in fact makes two separate, independently checkable requests. Each requirement is assigned a category (for example, naming, validation, testing, error handling, or API behaviour) and an ambiguity score. A request that a competent developer could not act on without further clarification is marked ambiguous and is never treated as verifiably satisfied, whatever the code subsequently does.

3.3 Evidence Generation

The commented file, as it exists at the end of the temporal window, is parsed with an incremental parser into a syntax tree, from which structural facts are derived: conditional expressions, function calls, raised or handled exceptions, return values, and test functions together with the inputs they exercise. A rule associated with the requirement's category then queries these facts to decide whether the requirement is met, without resorting to a machine-learning model for that decision. Where a rule cannot establish satisfaction with confidence — for instance, because the behaviour depends on a helper defined elsewhere — it returns an uncertain status rather than guessing. A machine-learned code-similarity model may additionally compute a relevance score between a requirement and the changed code; this score is recorded and shown as supporting evidence but never enters the decision itself.

3.4 Aggregation and Confidence Estimation

Per-requirement statuses are combined into one of four verdicts: fully satisfied, partially satisfied, not satisfied, or uncertain. A confidence value is attached to the verdict and is reduced whenever the temporal-window reconstruction flagged a reliability concern; the highest confidence level is withheld entirely pending a calibration procedure not yet completed for this system.

3.5 Explanation Generation

Every verdict is accompanied by an explanation that cites the specific structural facts it rests on, together with their file location and commit, so the reasoning can be checked independently of the tool that produced it.

3.6 Policy and Staged Enforcement

A policy stage turns the verdict and confidence into a recommended action — allow, human review, or block. An automated blocking action is enabled only when four independent conditions all hold at once, shown in Figure 3 (Section 4): a global switch; a per-repository operating stage that has progressed, one step at a time and only after a minimum monitoring period and a minimum number of reviewer confirmations, up to an enforcement stage; a determination that the requirement's category is statistically eligible, based on a one-sided upper confidence bound of both the false-acceptance rate and the false-blocking rate computed from a labelled evaluation set; and the repository's own configuration requiring the check to pass. Any one of these conditions being unmet is sufficient to keep the outcome non-blocking.

4. Verification Methodology

Figure 2 summarises the verification pipeline as a sequence of steps applied to a single review comment, from receipt of the comment to a recommended action.

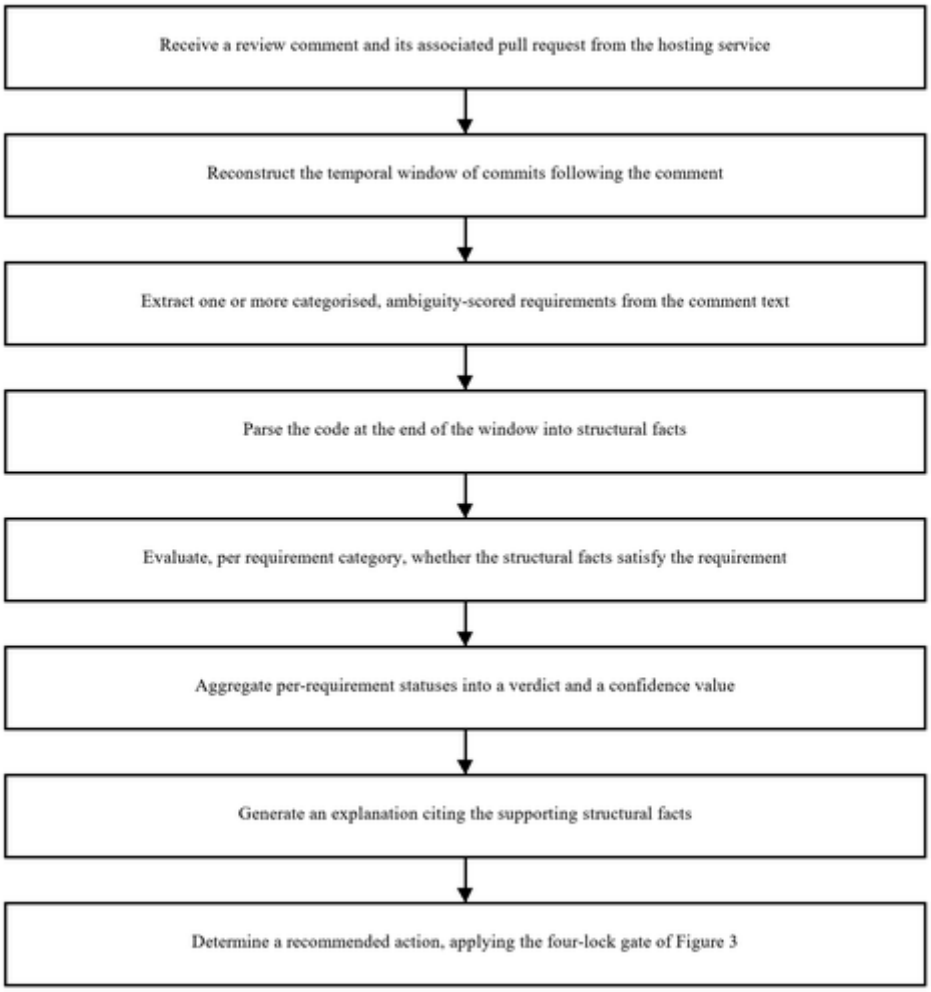


Figure 2: The verification pipeline.

Two design choices run through this pipeline. First, a deterministic rule always takes precedence over a machine-learned signal: this reflects the empirical finding, discussed in Section 7, that similarity- and relevance-based scoring accepts far more incorrect fixes than structural rule evaluation does. Second, whenever a rule cannot establish satisfaction with confidence, the outcome defaults to uncertain rather than to satisfied, since an unverified acceptance is the more costly mistake for a review-verification tool to make.

The pipeline also treats all text originating from the repository — the review comment, the code, and any commit message or thread reply within the temporal window — purely as data to be analysed, never as an instruction. This was tested directly by inserting adversarial instructional text (for example, text purporting to direct the system to mark a requirement satisfied) at several locations across the evaluation set; it produced no change in outcome.

Figure 3 shows how the four conditions of Section 3.6 combine to gate an automated blocking action.

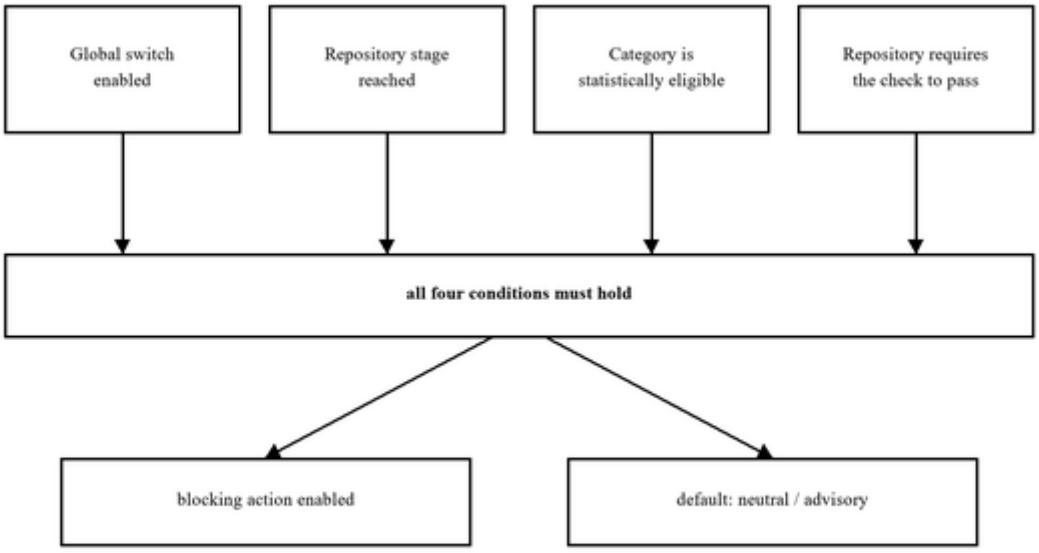


Figure 3: Conditions governing an automated blocking action.

5. Integration with GitHub (Advisory Mode)

The system integrates with a hosting platform through a webhook-driven advisory mode, illustrated in Figure 4. When a reviewer resolves a discussion thread, when a new commit is pushed to the pull request, or when a user asks for the check to be re-run, the hosting service sends a signed event to a webhook endpoint. The signature is verified over the raw, unparsed message body before any part of the message is parsed, and an unsigned or incorrectly signed message is rejected outright. A verified event is placed on a durable job queue keyed by a delivery identifier, so that a redelivered event is processed at most once. A worker process claims a queued task, obtains an access token scoped to the single repository and to the minimum permissions needed, and — critically — re-retrieves the thread and the code directly from the hosting service rather than trusting the content of the received message. Only then does it run the verification pipeline of Section 4, record an audit entry, and publish a neutral status check back to the hosting service.

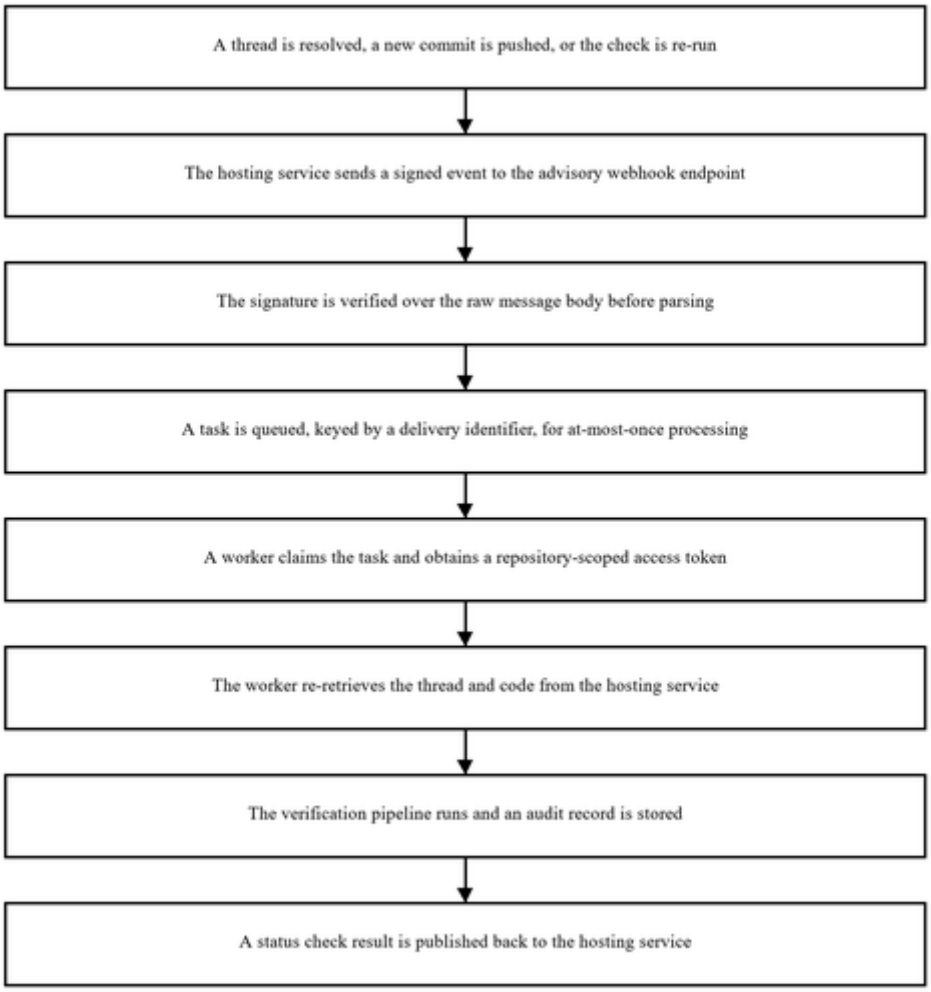


Figure 4: Advisory-mode event handling.

The integration was run live, as a GitHub App, on a private test repository on 8 October 2026. Review threads were resolved and fixes pushed by hand, and the check run, the audit trail, and the dashboard were observed end to end. The live run exposed seven problems that tests against recorded data had not:

- the worker produced no log output;
- a cited line number in an applied suggestion was offset by the function's start line;
- the platform's Re-run button sends a different event type than the one handled;
- ignored events were not logged;
- a check did not follow new commits;
- one unexpected error could stop the worker;
- tests shared the live database.

All seven were fixed and covered by tests. The check now re-runs on every push, by the operator's decision. The check stays neutral throughout: it reports, but cannot fail or block.

6. Dashboard and Operational Tooling

A read-only web dashboard lets an operator inspect, for a given repository, pull request, or individual verification, the verdict, confidence, explanation, the code before and after the temporal window, the diff, and the history of stage transitions and reviewer confirmations. The dashboard exposes no control that could itself change a verdict or a stage; such changes are made only through a separate, audited command-line interface. The dashboard is entirely disabled in the absence of a configured access token, sessions are bound to the token by a cryptographic check rather than by storing the token itself, and any repository-supplied text is rendered as plain, non-executable content regardless of what it contains. Stored source code associated with a verification is removed after a configurable retention period, while the verdict and explanation are kept as an audit trail.

7. Evaluation and Results

7.1 Protocol

The system was evaluated on benchmarks of three kinds: controlled cases written by hand; adversarial cases that plant traps such as a fix inside dead code, a mention of the fix in a comment only, or an unusual but valid idiom; and real-world review threads mined from six open-source Python projects. Each benchmark has a development split, used while building the rules, and a test split. The test split is frozen by cryptographic hash before any rule change and is evaluated exactly once, under a protocol written and hashed before the run. Once a test split has been run and its errors studied, it becomes development data, and the next round of changes is measured on a new test split frozen beforehand. Each later round runs the previous rules on the new test split as well, so old and new rules are compared case by case (paired bootstrap, 2,000 resamples, 95% intervals).

Table 2: The three blind evaluations, each run once. All results are measured.

Round	Test cases	Rules	Accuracy	False acceptance ↓	False blocking ↓	Real-world labels
1	120	initial	0.483 [0.39, 0.57]	0.237 [0.14, 0.35]	0.196	provisional (model)
2	147	first refinement	0.653 [0.58, 0.73]	0.151 [0.06, 0.26]	0.037	provisional when run; human since

3	148	second refinement	0.568 [0.49, 0.64]	0.088 [0.02, 0.17]	0.111	human (one annotator)

The rounds use different test cases, so their numbers are not directly comparable with each other; the paired comparisons within a round are. In round 2, the first refinement raised accuracy on the same 147 cases from 0.422 to 0.653 (+0.231, interval +0.16 to +0.31) and lowered false acceptance from 0.302 to 0.151 (−0.151, interval −0.25 to −0.07). Round 3 is discussed in Section 7.4.

7.2 Latest Results Against Baselines

Table 3 reports round 3 (60 controlled, 30 adversarial and 58 real-world cases), comparing the full system with text-similarity, code-model and structural baselines, and with the previous rule set on the identical cases.

Table 3: Round 3 blind test (148 cases). All results are measured; none are projected.

System	Accuracy	False acceptance ↓	False blocking ↓	Coverage
Lexical overlap	0.318	0.018	0.889	1.00
Sentence embeddings	0.304	0.000	0.931	1.00
Code-model similarity	0.507	0.825	0.097	1.00
Change-location baseline	0.480	0.982	0.014	0.99
Structural (AST) baseline	0.561	0.702	0.069	0.99
Requirements + AST, no rules	0.480	0.614	0.042	0.70
VeriReview, previous rule set	0.561 [0.48, 0.64]	0.105 [0.03, 0.19]	0.111	0.58
VeriReview, current rule set	0.568 [0.49, 0.64]	0.088 [0.02, 0.17]	0.111	0.57

VeriReview accepts far fewer incorrect fixes than the code-model, change-location and structural baselines (false acceptance lower by 0.53 to 0.90, all intervals excluding zero). The structural baseline matches its accuracy only by accepting almost every case. The two text-similarity baselines have a lower false-acceptance rate only because they reject almost everything, including 89–93% of correct fixes. When the system does decide, it is right 80% of the time; it leaves 43% of cases as uncertain for a human to judge. Verdicts given with medium confidence were right 82% of the time, and those with low confidence 26%.

On the 58 real-world cases, the system decided 13, 8 of them correctly, and accepted none of the resolutions the annotator judged unsatisfied. The other 45 were left for human review, mostly general refactoring, wording or style requests that no rule covers.

7.3 Human Labels

The real-world cases of the first two benchmarks were first labelled provisionally by a model, blind to the system's output. One human annotator then relabelled the 60 real-world cases of round 2, blind, and labelled the 60 fresh real-world cases of round 3 before any verifier ran on them. Agreement between the human and the provisional labels was a Cohen's kappa of 0.60. The provisional labels had been lenient: the human judged 7 resolutions unsatisfied where the model

had judged 3. Re-scoring the recorded round 2 predictions against the human labels, without running the system again, showed that one false acceptance had been hidden by the provisional labels (bad resolutions accepted: 0 of 3 before, 1 of 7 after).

A second annotator, the project owner, then labelled 36 of the round 2 real-world cases blind. These were the 21 cases on which the human and the provisional labels disagreed, plus 15 drawn at random. Because the owner had seen earlier system results, this second opinion is not fully independent. On the 15 random cases the two humans agreed on all 15, but these were easy cases, 14 of them plainly satisfied. On the 21 disputed cases they agreed on only 5. Pooled over the 36, Cohen's kappa on the verdict was 0.33, below the target of 0.6. The largest single difference was whether a comment asks for anything at all: one annotator excluded 8 cases on that ground, the other none.

The two annotators settled their 19 disagreements together, each with a written reason. One decision was later revised on re-check, with knowledge of the system's verdict, and is recorded as such. Re-scored against the adjudicated labels, the recorded round 2 predictions accept none of the 4 bad real-world resolutions. The earlier apparent false acceptance is now judged correct. With so few bad cases this is weak evidence, not a demonstration of safety. The broader finding is that, for hard cases, what counts as “satisfied” is often genuinely disputed between careful human readers. This bounds how precisely any verifier, human or automatic, can be measured on such cases.

7.4 A Second Round of Refinement, and Why It Barely Transferred

The second round of rule refinement was developed on the development data only (345 cases, including the human-labelled round 2 cases). It targeted the false acceptances analysed after round 2:

- a check, handler or response that can never run;
- an HTTP exception created but never raised;
- a validation condition written the wrong way round;
- a check that only runs behind an unrelated flag;
- a test in a file that test runners do not collect;
- a test that never shows the scenario the reviewer asked about.

It also added two common real-world request forms: statements that code is no longer needed, and requests for an exact comment. Every change has positive, negative and adversarial tests. On the development data the effect was large: false acceptance fell from 6.1% to 0.7%, and 11 cases were fixed with none broken.

On the round 3 test, however, only 2 of 148 verdicts changed: two former acceptances became uncertain. Accuracy rose by 0.007 (interval 0.00 to +0.02, not a clear improvement), and false acceptance fell by 0.018 (interval −0.06 to 0.00). The pre-registered safety criterion, that false acceptance must not rise, was met. The improvement criterion was not. The gap between development and test accuracy grew from 0.145 in round 2 to 0.203. The finding is that a fix written for one error pattern seen in development rarely meets the same pattern among new cases. The remaining errors on round 3 are of new kinds:

- a rename completed in one function but not in the rest of the module;
- a second request in the same comment judged by the first;
- a test whose wording matches the requested scenario while its data does not;
- a check placed under a block that never runs at run time.

7.5 Robustness and Requirement Extraction

A separate evaluation planted adversarial instructional text at multiple locations in every development and held-out case (1,288 injected variants per verifier) and found no resulting change in outcome, before and after both rounds of rule refinement. This supports the treat-content-as-data design discussed in Section 4.

Requirement extraction, evaluated separately against a blind, hand-labelled set of comments, reached an exact requirement count of 0.844 and a category F1 of 0.909, against targets of 0.80 and 0.85 respectively.

8. Toward the Complete System: Targets and Projected Outcomes

Everything reported in Section 7 was measured. This section instead describes the complete, fully deployed system that the current design is working toward, and states the quantitative thresholds it is required to reach at each remaining step. These are design targets already built into the system's own gating logic (Section 3.6), not results obtained from running the system — the further work described below has not yet been carried out, and no accuracy figure beyond those in Section 7 should be read as measured.

8.1 From One Annotator to Adjudicated Labels

The complete system relies on labels from two annotators working independently and blind to each other, with disagreements adjudicated. The labelling, agreement and adjudication workflow has now been used end to end on 36 cases (Section 7.3). The measured agreement, a kappa of 0.33, is below the target of 0.6 set when the evaluation protocol was designed. Two causes stand out. The first is the exclusion rule: whether a comment such as an empty suggestion or a “not blocking” remark asks for anything. The second is how much of a request must be done to count as satisfied. The complete system therefore needs the annotation guide sharpened on exactly these points, with worked examples. Two fully independent annotators should then label every new real-world case, and kappa should be measured again on a fresh random sample before the labels are trusted as ground truth.

8.2 Data Volume Required for Statistical Enforcement Eligibility

Section 3.6 permits an automated blocking action for a requirement category only once the one-sided 95% upper confidence bound of both its false-acceptance rate and its false-blocking rate, computed from a frozen test run, falls at or below 5%. With zero observed errors, that bound is approximately 3 divided by the number of cases evaluated. Roughly 59 clean cases on each side (bad resolutions correctly rejected, and good resolutions correctly accepted) are therefore needed per category before the bound can reach 5%. The round 3 test provides, per rule category, 10 to 12 bad and 7 to 14 good resolutions. The observed false-acceptance rates there are 0% to 20%, with upper bounds of 22% to 51%. No category is eligible, short on both volume and, for most categories, accuracy. Table 4 states the targets explicitly.

Table 4: Current measurements against the complete system's defined targets.

Metric	Current (measured)	Target for the complete system
Agreement between two human annotators (Cohen's κ)	0.33 on 36 cases (random 15: 15 of 15 agree; disputed 21: 5 of 21); 19 disagreements adjudicated	≥ 0.6 on the four-way verdict
Labelled cases per requirement category	10–12 bad and 7–14 good (round 3 test)	≥ 59 bad and ≥ 59 good cases (needed for the 5% bound below)
False-acceptance rate, per category, 95% upper bound	22%–51% (round 3 test)	≤ 5%, required before that category may ever be enforced
False-blocking rate, per category, 95% upper bound	30%–60% (round 3 test)	≤ 5%, required before that category may ever be enforced

Categories eligible for enforcement	0	computed automatically once the above are met; not set by hand
Live deployment	run end to end on one private test repository (October 2026)	running continuously against at least one real, active repository

Reaching this volume means mining and labelling on the order of several hundred additional real-world cases from the currently approved repositories, using the same collection, pseudonymisation and labelling tools already built for the existing benchmarks. No category is assumed to pass this threshold in advance. The eligibility computation is automatic and would be re-run once the new data exists; it could just as well continue to exclude a category.

8.3 Live Deployment

The advisory integration of Section 5 has run end to end on a private test repository, with review activity created for the test. The complete system runs it on at least one real, active repository for a sustained period. The dashboard of Section 6 would then reflect genuine reviewer activity, and the false-acceptance and false-blocking rates of Table 4 could eventually be estimated on live, rather than mined, cases.

8.4 Direction of Future Refinement

The first round of rule refinement transferred well to new cases (Table 2, round 2); the second did not (Section 7.4). The next step is therefore a change of method rather than more rules of the same kind. It means collecting a larger set of fresh, human-labelled real-world cases, studying which request forms recur across projects, and writing rules only for forms common enough to matter on new data. Consistent with the protocol followed throughout, any such change would be validated on a fourth test split frozen before the change. No numeric result is claimed for that round in advance.

8.5 Coverage of Real-World Requests

The system decided 44% of the round 2 real-world cases, nearly all through the check for suggestion blocks, and 22% (13 of 58) of the round 3 cases, with no false acceptance in either. Most real review requests are general refactoring, wording or style requests outside the categories the rules cover; these are left as uncertain by design. The complete system is expected to raise the decided fraction while keeping false acceptances on real-world cases at zero. This, too, is a target to be verified on new data, not a result already obtained.

9. Threats to Validity and Replication

9.1 Construct Validity

Whether a request is “satisfied” is itself open to interpretation. Two annotators agreed on all 15 randomly drawn cases but on only 5 of 21 already-disputed ones (kappa 0.33 over 36). Most disagreements concerned whether a comment asks for anything at all, and how much of a request must be done. The verdict is derived from per-request statuses, so a label cannot contradict itself, and all disagreements were adjudicated with written reasons. False acceptance is the primary metric because it captures the costly error. False blocking, coverage and selective accuracy are reported beside it, because counting “uncertain” as wrong understates the value of abstaining. The verifier sees only the commented file after the change; a fix made elsewhere is out of scope by design.

9.2 Internal Validity

The controlled and adversarial test cases were written by the author of the rules. This happened before each round's rule changes, from a fixed recipe whose proportions are checked by tests, but targeting cannot be ruled out; the real-world cases are the independent part. The real-world labels of rounds 1 and 2 were provisional model labels when those rounds ran; they were later replaced by human labels, and the recorded runs were re-scored rather than re-run. The second annotator was the project owner and not fully independent, and one adjudicated decision was revised with knowledge of the system's verdict; this is recorded as such. Leakage from test to development data is controlled: each test set was frozen by hash before rule changes, run once under a hashed protocol, then retired. Baseline thresholds come from development data only.

9.3 External Validity

The real-world cases come from six permissively licensed open-source Python projects. Other languages, industrial settings and review cultures may differ. Coverage depends on the mix of requests: GitHub suggestion blocks, which are easy to check, appear in 28 of 60 round-2 real-world comments but in only 12 of 60 in round 3, and real-world coverage fell from 0.44 to 0.22. Most resolved threads were in fact addressed, so the real-world false-acceptance rate rests on few bad cases (7 in round 3, none accepted). The live run covered one private test repository; no user study with developers was carried out.

9.4 Conclusion Validity

The test sets are small (120, 147 and 148 cases), so intervals are wide; for example, round-3 false acceptance is 0.088 with a 95% interval of 0.02 to 0.17. Every main number carries a bootstrap interval (2,000 resamples, fixed seed), and comparisons are paired on identical cases. Up to three threads may come from one pull request, and resampling cases rather than pull requests may make the intervals slightly too narrow. Each round was run once, so variance over rule versions or annotators is not measured. Only the pre-registered primary questions are treated as confirmatory; the ablation rows are descriptive. The enforcement gate uses exact, conservative Clopper–Pearson bounds, and no category passes it.

9.5 Reliability and Replication

The pipeline is deterministic: no language model decides, and the code model is pinned to a revision and kept neutral. All test sets and human labels are pinned by hash in the test suite, raw annotation exports are never edited, and every label change is logged. Each of the three recorded blind test runs was re-run from the commit at which it was recorded. The predictions, gold labels and confidences were identical for every case (120 of 120, 147 of 147 and 148 of 148), and so were the bootstrap intervals. The round-3 code matches the source hash pre-registered in its protocol. The repository serves as the replication package: code, frozen benchmarks, human labels, protocols, every recorded report and step-by-step instructions (REPLICATION.md).

9.6 Limitations

- Not accurate enough to gate merges on its own: on the latest blind test, 9% of bad resolutions were still accepted and 11% of good ones were judged unsatisfied. The system is positioned as an advisory signal for human review, not a replacement for it.
- Rule refinement transfers poorly: the second round fixed 11 development cases but changed only 2 of 148 test verdicts (Section 7.4).
- Most real review requests fall outside the categories the rules cover and are left as uncertain, by design, rather than guessed at (Section 8.5).
- The rules reason over program structure within a single file and one level into same-file helper functions; they do not perform whole-program control-flow analysis.
- The implementation covers a single programming language and one commented file per review case.

10. Conclusion

This work shows that a review comment's resolution can be checked more safely by verifying program structure against a categorised requirement than by scoring textual or semantic similarity, and that a machine-learned relevance signal is more useful as supporting evidence than as the basis for a decision. Across three blind evaluations, each run once, the system's false-acceptance rate fell from 24% to 15% to 9%. The first round of refinement clearly improved the same test cases; the second, measured just as strictly, did not. That negative result is as informative as the positive one. Hand-written fixes for individual error patterns reach a limit, and further progress depends on more and better labelled real-world data. Throughout, any automated blocking action stays gated behind conditions that must be independently and statistically justified before they can take effect. Section 8 sets out, explicitly as targets rather than results, what remains: a sharper annotation guide and sufficient agreement between independent annotators, enough labelled volume per category to justify enforcement, a sustained live deployment, and refinement validated on a newly frozen test set.

References

- Beller, M., Bacchelli, A., Zaidman, A., and Juergens, E. (2014). Modern Code Reviews in Open-Source Projects: Which Problems Do They Fix? Proceedings of MSR 2014.
- Bosu, A., Greiler, M., and Bird, C. (2015). Characteristics of Useful Code Reviews: An Empirical Study at Microsoft. Proceedings of MSR 2015.
- Clopper, C. J., and Pearson, E. S. (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4), 404–413.
- CodeRabbit. Request changes workflow (documentation). https://docs.coderabbit.ai/pr-reviews/request-changes-workflow.
- Falleri, J.-R., Morandat, F., Blanc, X., Martinez, M., and Monperrus, M. (2014). Fine-grained and accurate source code differencing. Proceedings of ASE 2014, 313–324.
- Falleri, J.-R., and Martinez, M. (2024). Fine-grained, accurate and scalable source differencing. Proceedings of ICSE 2024.
- Fluri, B., Wuersch, M., Pinzger, M., and Gall, H. (2007). Change Distilling: Tree Differencing for Fine-Grained Source Code Change Extraction. IEEE Transactions on Software Engineering, 33(11), 725–743.
- GitHub (2026). Auto-resolution and analysis updates in Copilot code review. GitHub Changelog, 11 September 2026.
- GitHub, Inc. GitHub REST API and GraphQL API documentation. https://docs.github.com/en/rest and https://docs.github.com/en/graphql.
- Google Research (2023). Resolving code review comments with ML. Google Research Blog, 23 May 2023.
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., and Yin, J. (2022). UniXcoder: Unified Cross-Modal Pre-training for Code Representation. Proceedings of ACL 2022.
- Li, Z., Lu, S., Guo, D., Duan, N., et al. (2022). Automating Code Review Activities by Large-Scale Pre-training (CodeReviewer). Proceedings of ESEC/FSE 2022. arXiv:2203.09095.
- Panthaplackel, S., Li, J. J., Gligoric, M., and Mooney, R. J. (2021). Deep Just-In-Time Inconsistency Detection Between Comments and Source Code. Proceedings of AAAI 2021, 427–435.
- Rahman, M. M., Roy, C. K., and Kula, R. G. (2017). Predicting Usefulness of Code Review Comments using Textual Features and Developer Experience. Proceedings of MSR 2017, 215–226.
- Ramirez, S. FastAPI. https://fastapi.tiangolo.com/.
- Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019.
- Sun, K., Kuang, H., Baltes, S., Zhou, X., Zhang, H., Ma, X., Rong, G., Shao, D., and Treude, C. (2025). Does AI Code Review Lead to Code Changes? A Case Study of GitHub Actions. arXiv:2508.18771.
- Tree-sitter. An incremental parsing library. https://tree-sitter.github.io/tree-sitter/.
- Tufano, R., Pascarella, L., Tufano, M., Poshyvanyk, D., and Bavota, G. (2021). Towards Automating Code Review Activities. Proceedings of ICSE 2021. arXiv:2101.02518.